



Formulário de orientação e realização da Produção Temática

SELEÇÃO ESPECÍFICA PSI N. 15339

Nome: Thiago da Silva Menezes

Matricula: C137459-9

1. UNIDADE DEMANDANTE

Código	Nome
5229	GN CAPACIDADE E PADROES

2. UNIDADE(S) DE PROVIMENTO

Código	Nome
5229	GN CAPACIDADE E PADROES

3. FUNÇÃO GRATIFICADA

Código	Nome
2030	COOR PROJETOS/PROCESSOS MATRIZ

4. ORIENTAÇÕES PARA A PRODUÇÃO TEMÁTICA – TEMA/ FORMATAÇÃO/ INFORMAÇÕES ADICIONAIS

⇒ Esta produção temática possui um item avaliativo, totalizando 100 pontos. Leia atentamente as instruções de cada item e respeite o formato obrigatório de entrega.

Item	Tema	Entrega	Peso
1	Proposta de atuação de um integrador de código.	Documento Word ou PDF	100

1. Contexto

Em ambientes de desenvolvimento de software, especialmente em contextos de grande escala e alta criticidade, a existência de um processo estruturado de controle de versão e integração de código é condição fundamental para garantir controle, previsibilidade, rastreabilidade e confiabilidade nas entregas.

Sem um processo claro, baseado em práticas consistentes de gestão de branches, integração contínua, versionamento semântico e validação de mudanças, o repositório tende a se degradar, passando a refletir inconsistência, perda de rastreabilidade e aumento do risco operacional. Nesse cenário, não há garantia de que as versões promovidas representem efetivamente código íntegro, testado e aderente aos princípios de qualidade e segurança.

Diante dessa necessidade, emerge o papel do Integrador como elemento central de governança técnica. Sua atuação não se limita à execução operacional de merges, mas consiste em assegurar que todo o fluxo de evolução do código ocorra dentro de um processo disciplinado, controlado e auditável.

Espera-se do integrador a capacidade de preservar a integridade das branches permanentes, garantir que toda alteração percorra um fluxo de integração formal, aprovar versões candidatas, evitar paralelismos indevidos e promover a adoção consistente de práticas institucionais de versionamento e integração de código.

Sua atuação deve refletir consistência, critério técnico e capacidade de organização em escala, garantindo que o repositório seja mantido como um ativo confiável e governável, mesmo em contextos de pressão por entrega e baixa maturidade dos times.

2. Cenário

Você atua como Integrador de código responsável por diversos repositórios utilizados por diferentes linhas de desenvolvimento e times, cada um com certa autonomia operacional e diferentes níveis de maturidade técnica.

Nesse contexto, você é acionado para estruturar e sustentar a governança de versionamento e integração de código em escala, garantindo que o processo de desenvolvimento seja confiável, auditável e aderente às diretrizes institucionais.

A iniciativa consiste em assegurar que todo o fluxo de evolução do código, desde o desenvolvimento até a disponibilização em produção, ocorra de maneira controlada, consistente e previsível.

O cenário apresenta desafios relevantes, como uso inconsistente de práticas de branch e versionamento, integrações tardias com conflitos frequentes, ausência de padronização em commits e versionamento, repositórios com histórico pouco confiável, atuação descentralizada dos squads e pressão por entregas rápidas que frequentemente tensionam o processo.

Adicionalmente, há a necessidade de garantir conformidade com requisitos institucionais, o que exige disciplina operacional, controle rigoroso das integrações e garantia de integridade do histórico.

3. Produção Temática

A partir do contexto apresentado, elabore um texto estruturado no qual você descreva como atuaria como Integrador para garantir o controle de versionamento e a integração de código em um ambiente de grande escala.

Sua redação deve apresentar de forma articulada e consistente as práticas que você adotaria para organizar o fluxo de evolução do código, bem como a estratégia que utilizaria para estruturar e sustentar esse processo no dia a dia dos times.

Ao longo do texto, deve ficar claro como você estruturaria o processo de integração de código, especialmente por meio do uso de pull requests, critérios de aprovação, proteção das branches permanentes e disciplina de integração contínua. Também se espera que você aborde a adoção do GitFlow, descrevendo como aplicaria esse modelo para organizar o fluxo de branches, incluindo o tratamento adequado de correções emergenciais por meio de hotfix e a necessidade de reconciliação do código com as demais linhas de desenvolvimento.

Sua abordagem deve contemplar ainda como você atuaria para manter as branches próximas ao longo do tempo, evitando desvios significativos entre elas, bem como como trataria a clareza e a qualidade do histórico do repositório, incluindo práticas relacionadas a commits, versionamento e identificação de versões, garantindo compreensão consistente da evolução do código.

Adicionalmente, descreva como atuaria para evitar a degradação do repositório ao longo do tempo, especialmente em situações de alta pressão por entrega, paralelismo e divergência entre equipes, e como sua atuação contribui para a consolidação e evolução das práticas adotadas, promovendo consistência de comportamento entre os times e elevação gradual do nível de maturidade.

Por fim, espera-se que sua redação evidencie como sua atuação permite demonstrar, na prática, que o processo está funcionando, considerando sinais observáveis no próprio repositório, como alinhamento entre branches de referência, qualidade dos pull requests, consistência das versões e previsibilidade das entregas.

Seu texto deve refletir clareza de raciocínio, domínio técnico e capacidade de estruturar, sustentar e evoluir o processo de versionamento e integração de código em escala.

3.1. O que você deve abordar

A. Compreensão do problema e seus impactos

Capacidade de enquadrar o problema como uma questão de governança de versionamento e integração, evidenciando riscos como perda de controle do fluxo, divergência entre branches, baixa rastreabilidade e imprevisibilidade das entregas.

B. Estratégia de atuação do integrador

Clareza na definição da atuação como Integrador, evidenciando as práticas adotadas, a organização do fluxo (incluindo GitFlow) e a forma como o processo é estruturado e sustentado ao longo do tempo.

C. Aplicação prática e operacionalização

Capacidade de explicar como as práticas são implementadas no dia a dia, incluindo uso de pull requests, controle de branches, tratamento de integrações, hotfix e reconciliação (merge back), bem como manutenção de branches próximas (ahead/behind).

D. Capacidade de garantir e sustentar o processo

Demonstração de como o candidato assegura que o processo seja seguido, por meio de mecanismos de controle, validação, automação e acompanhamento, evitando desvios e degradação do repositório.

E. Atuação em contexto matricial e evolução de maturidade

Forma como o candidato influencia os times, promove consistência de comportamento, dissemina práticas e contribui para a evolução do nível de maturidade, mesmo sem autoridade hierárquica direta.

F. Qualidade, governança e confiabilidade do repositório

Capacidade de garantir integridade das branches, clareza do histórico, consistência de commits e versões, e aderência às práticas institucionais.

G. Evidências e validação do processo

Apresentação de sinais objetivos de que o processo está funcionando, com base no próprio repositório, como organização das branches, proximidade entre elas, qualidade dos pull requests, consistência das versões e previsibilidade das entregas.

3.2. Formato e limites

- **Extensão sugerida:** 900 a 1.500 palavras (aprox. 2 a 3 páginas)
- **Estrutura:** texto em formato de redação estruturada, com progressão lógica clara
- **Organização:** uso de títulos e eventuais bullets para apoiar a leitura, sem prejudicar o encadeamento do raciocínio
- **Abordagem:** explicitar o que irá fazer, como irá fazer e como irá garantir a execução e a sustentação das melhores práticas de versionamento e integração de código.
- **Contexto:** tratar como cenário hipotético, sem uso de informações confidenciais

5. ELABORAÇÃO DA PRODUÇÃO TEMÁTICA



1. Enquadramento Estratégico: O Repositório como Ativo de Governança

Em ecossistemas corporativos de grande escala e alta criticidade — com múltiplas linhas de negócio, dezenas de repositórios e esquadrões (*squads*) heterogêneos —, a ausência de versionamento e integração estruturados transcende o gargalo operacional: **configura uma falha grave de governança**. Quando cada equipe decide isoladamente como nomear ramificações, integrar artefatos ou rotular pacotes, o repositório perde a confiabilidade e espelha a dispersão do processo.

Os sintomas sistêmicos são severos: instauração do “*inferno do merge*” (*merge hell*), divergência paralela acumulativa, reintrodução de regressões, histórico irracional e binários promovidos sem correspondência determinística com o código-fonte. Em instituições sujeitas a rigorosas regulamentações bancárias e auditorias contínuas — que exigem rastreabilidade ponta a ponta entre a motivação negocial, a alteração sintática, a revisão técnica e o executável —, essa desordem traduz-se em risco operacional inaceitável.

Neste cenário, o papel do Integrador de Código supera a mecânica de comandos de junção. Atuo como **guardião da “Fonte Única da Verdade”** (*Single Source of Truth*), suportando estrategicamente a liderança técnica para garantir que a evolução do software ocorra em um ecossistema disciplinado, previsível, automatizado e auditável.

2. Topologia de Repositórios e Arquitetura de Versionamento

Para ordenar o portfólio, a primeira medida do Integrador consolida a topologia de repositórios, segregando lógicas de aplicação (Backend BFFs, APIs Rest/GraphQL e Micro Frontends) de repositórios paralelos de configuração e Infraestrutura como Código (IaC, para contêineres e manifestos). Toda a base adotará a convenção estrita `<projeto>-<modulo>-<funcionalidade>[-iac][-ambiente]`.

A estratégia regente será o **GitFlow Híbrido Adaptado**, estruturado em:

- **main**: Linha permanente, intocável e protegida, refletindo linearmente o código em Produção.
- **develop**: Tronco permanente de entrega contínua para consolidação da próxima liberação (*Release*).
- **feature/***: Ramificações efêmeras nascidas estritamente da develop para novas capacidades, amarradas a um item de trabalho.
- **release/***: Ramificações de estabilização isoladas da develop ao atingir a maturidade de candidata à liberação (*Release Candidate*).
- **hotfix/***: Ramificações emergenciais criadas exclusivamente da main para sanar falhas críticas em produção.

A mesma topologia aplica-se aos repositórios de infraestrutura, sincronizando a evolução de aplicação e ambiente.

3. Aplicação Prática e Operacionalização do Fluxo de Valor

No cotidiano dos times, a operacionalização assenta-se em quatro pilares inegociáveis:

3.1. O Rito do Pull Request (PR) como Porta Única

Fica vedado o envio direto (*direct push*) para develop ou main. Toda mesclagem exige a abertura de um PR na plataforma corporativa, atuando como fórum de governança e validação. Institui-se um *template obrigatório* (exigindo descrição de impacto, plano de testes e chave de rastreabilidade do item de trabalho) aliado ao mecanismo de **CODEOWNERS**, que aciona sistemicamente os líderes técnicos do componente alterado.

3.2. Revisão de Código Automatizada via IA (Shift-Left AI Review)

Para antecipar a detecção de vulnerabilidades e débitos técnicos antes da alocação de revisores humanos, institui-se a revisão automatizada via **GitHub Copilot Code Review**. Através de *Branch Rulesets* globais (Automatically request Copilot code review), o agente de IA é acionado a cada PR aberto ou novo *commit*. Operando sobre a infraestrutura do GitHub Actions, a IA atua como revisor **Tier-0**, varrendo o código atrás de *bugs*, lógicas confusas, duplicidades, falta de tratamento de erros e desvios de legibilidade. Crucialmente, sua análise é registrada estritamente sob o status de **Comment** (sem emitir Approve ou Request changes). Assim, a IA acelera o feedback e limpa o código na origem (*shift-left*), mas não conta como aprovação obrigatória nem bloqueia a mesclagem, preservando a autoridade soberana do revisor humano e dos qualificadores estáticos.

3.3. Runbook de Hotfix e Reconciliação Obrigatória (Merge Back)

O tratamento de emergências segue um *runbook* com SLA rigoroso de até 60 minutos. A hotfix/<ID-Incidente> nasce da main com a correção estritamente necessária. Para garantir a **proibição de refatorações de escopo**, institui-se uma governança dupla:

- **Supervisão Semântica (Alarme):** Inspeccionado via *GitHub Copilot Code Review*, o *diff* dispara um alerta técnico assíncrono caso detecte refatorações estruturais desacopladas do incidente.
- **Garantia Determinística (Trava):** Como a IA emite revisões não-bloqueadoras (Comment), a trava absoluta contra o “contrabando de código” é delegada a um *Status Check* volumétrico no GitHub Actions (*Hotfix Scope Gatekeeper*). Ele aplica um exit 1 (bloqueando o *merge*) caso a requisição viole limites matemáticos de arquivos/linhas ou altere diretórios de infraestrutura.

Após a validação em produção, executa-se a reconciliação obrigatória (*merge back*) para a develop e linhas *release/** ativas. O rito é indissociável; impede a reintrodução da falha.

3.4. Mitigação de Desvios Estruturais (Ahead/Behind Drift)

Para impedir o distanciamento silencioso entre as linhas de configuração, estabilização e o tronco principal, impõe-se a sincronização contínua por rebaseamento (*git pull --rebase*). Complementarmente, uma automação calcula semanalmente a defasagem de *commits (ahead/behind)* do portfólio. Desvios severos disparam alertas em painéis, acionando reconciliações programadas ou o descarte de ramificações abandonadas.

4. Sustentação Sistêmica e Esteiras de Automação

A disciplina técnica em larga escala exige barreiras de proteção sistêmicas (*guardrails* automatizados):

4.1. Travas de Proteção de Ramificações (Branch Protection Rules)

Configuradas via API corporativa, as regras para main e develop impõem: bloqueio de mesclagem sem PR aprovado por revisores qualificados, proibição de deleção ou envio forçado (*no-force-push*) e ativação de **dismiss stale reviews**, descartando aprovações prévias se novos *commits* forem injetados.

4.2. Portões de Qualidade e Segurança (Quality Gates)

A mesclagem de um Pull Request está tecnicamente vinculada à aprovação integral em **fluxos de trabalho padronizados e reutilizáveis** de DevSecOps — capacidade tecnológica já implementada e consolidada corporativamente pelo CoE de Nuvem e DevSecOps —, que invocam sistemicamente:

- **Segurança Contínua (SAST/SCA):** Varredura via analisadores estáticos avançados (CodeQL) e checagem de dependências, aplicando bloqueio sumário diante de vulnerabilidades Alta/Crítica ou presença de senhas fixas no código (*secret scanning*).
- **Qualidade de Código:** Inspeção de regras sintáticas e métricas (SonarQube), restando o pacote caso a nota geral seja inferior a Grade A/B ou a **cobertura de testes unitários automatizados seja inferior à régua de 80%**.
- **Documentação Contínua:** Compilação de documentação técnica em Markdown via MkDocs. O artefato é validado e publicado no portal de documentação interna a cada mesclagem, unindo a especificação ao software.

4.3. Rastreabilidade Operacional Integrada

Para assegurar paridade entre a gestão estratégica e a engenharia, todo envio deve atrelar-se a um *Workitem* do módulo corporativo de gestão de portfólio (ServiceNow SPM) — **evoluindo a rastreabilidade regida historicamente pelo RTC** —, categorizado como Demanda (iniciativa estratégica) ou Defeito refinado para execução ágil.

5. Atuação Matricial e Estágios de Maturidade (E1–E5)

Sem autoridade hierárquica direta sobre os esquadrões, o Integrador consolida a governança por **influência estruturada**, facilitação e um modelo pautado por cinco estágios evolutivos de maturidade:

- **E1 (Padronização Básica):** *Branch protection* em main e develop, erradicação de envios diretos, PR obrigatório e passagem pelos *status checks* essenciais.

- **E2 (Higiene de Histórico):** Conventional Commits via *commitlint*, mapeamento de CODEOWNERS, vínculo a itens de trabalho e compactação linear (*squash and merge*).
- **E3 (Versionamento e Reconciliação):** Versionamento semântico maduro, geração automatizada de *Release Notes*, *runbook* de *hotfix* com SLA cumprido e *merge back* obrigatório.
- **E4 (Sustentação por Telemetria):** Monitoramento automatizado *ahead/behind*, estabilização do *First Time Right* (PRs aprovados na primeira tentativa) > 90% e rastreabilidade integral de auditoria.
- **E5 (Excelência Evolutiva):** Esquadrões com autonomia plena, Integrador focado em engenharia de plataforma, testes contratuais e entrega progressiva (*progressive delivery*).

A transição é apoiada por gabaritos e automações de autoatendimento no Portal Interno do Desenvolvedor (IDP), *Dojos* técnicos e mentorias no *Code Review*. Sob pressão por entregas, a postura mantém-se firme: acelera-se a revisão humana, mas **nunca se suprime a passagem pelos Quality Gates automatizados**.

6. Governança Semântica e Confiabilidade do Repositório

A confiabilidade do repositório reflete-se na capacidade de um engenheiro recém-alocado compreender a evolução do produto inspecionando exclusivamente a árvore de *logs*. Para tanto, impõe-se:

- **Higiene Semântica de Mensagens:** Prefixos padronizados (*feat*:, *fix*:, *docs*:, *refactor*:, *test*:, *ci*:). Descrições vagas são barradas na origem pelo *commitlint*, viabilizando a geração limpa do *Changelog*.
- **Linearidade Estrutural:** Na mesclagem de funcionalidades na *develop*, aplica-se a compactação de histórico (*Squash and Merge*), condensando dezenas de envios em um único *commit* atômico. Na promoção para a *main*, utiliza-se o *Merge Commit* assinado criptograficamente, preservando a assinatura da liberação.
- **Versionamento Semântico Imutável (SemVer):** Abolição de sufixos instáveis (SNAPSHOT). Toda liberação produtiva recebe uma marcação estática (*Git Tag* vX.Y.Z) atrelada ao hash SHA-1 do *commit*, garantindo paridade absoluta com a Requisição de Mudança formal (submetida com os 10 dias regulamentares de antecedência).

7. Evidências Objetivas de Validação do Processo

A excelência operacional da Gerência de Configuração e Mudança comprova-se por sinais observáveis na telemetria nativa do repositório:

1. **Alinhamento Topológico:** O gráfico de ramificações exibe a *develop* avançando em estrito paralelo à *main*, sem desvios silenciosos (*drift zero*), provando a execução contínua do *merge back*.
2. **Qualidade na Origem (Shift-Left):** O *First Time Right* consolida-se acima de 90%, provando que a quase totalidade dos PRs atinge a mesclagem com a esteira verde no primeiro envio, impulsionada pelo filtro prévio de IA.
3. **Estabilidade da Base Promovida:** Erradicação de *commits* de “correção rápida” disparados após o fechamento de uma liberação, comprovando a eficácia dos portões de qualidade.
4. **Previsibilidade de Liberação:** Implantações produtivas fluem de forma determinística, cumprindo os cronogramas de gestão de mudanças, com taxa nula de *rollback* por falha de empacotamento.

Através dessa abordagem — que amarra arquitetura de ramificações, assistência de IA, barreiras sistêmicas, automação corporativa e aculturação humano —, o Integrador consolida o repositório como um ativo governável, previsível e de altíssima confiabilidade.